



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

**Intelligent Spell Checker using Levenshtein Edit
Distance**

NUGURI SIRI

2520030309

M.Geethika

2520030485

COURSE INSTRUCTOR

Dr. J Sirisha Devi

Professor

Department of Computer Science and Engineering

TITLE OF THE PBL

Intelligent Spell Checker using Levenshtein Edit Distance

Brief Description of the Case Study

1. Introduction

Spelling mistakes are very common when people type messages, assignments, documents, emails, reports, and other digital content. A small spelling mistake can change the appearance and readability of a sentence and may sometimes make the meaning difficult to understand. As people increasingly depend on computers and mobile devices for writing, there is a need for systems that can automatically identify spelling mistakes and suggest suitable corrections.

The case study presented in this project focuses on developing an **Intelligent Spell Checker using Levenshtein Edit Distance**. The system is designed to identify incorrectly spelled words and provide the user with a list of possible correct words. Instead of only telling the user that a word is incorrect, the system tries to find the closest matching words from a dictionary.

For example, if the user enters:

"I am studnet."

The word **"studnet"** is not present in the dictionary. The spell checker identifies it as an incorrect word and compares it with dictionary words. It can identify **"student"** as a close match and provide it as a correction.

Similarly, if the user enters:

"I like programing."

The system can identify "**programing**" as a possible spelling mistake and suggest "**programming**".

The main algorithm used in this project is **Levenshtein Edit Distance**, which calculates the minimum number of operations required to convert one word into another. The three basic operations are **insertion, deletion, and substitution**. The algorithm is implemented using **Dynamic Programming**, which stores previously calculated results and reuses them to avoid unnecessary repeated calculations.

2. Problem Statement

Traditional dictionary-based spell checkers can determine whether a word exists in a dictionary or not. However, simply identifying an incorrect word is not always sufficient. The user also needs to know what the correct word could be.

For example, consider the word:

"recieve"

A basic dictionary lookup can determine that the word is not present in the dictionary. However, it does not directly explain which word the user intended to type. A spell correction system can compare the incorrect word with dictionary words and identify:

recieve → receive

Therefore, the problem addressed by this case study is to develop a system that can:

- Detect incorrectly spelled words.
- Compare incorrect words with dictionary words.
- Calculate the similarity between words.

- Generate possible corrections.
- Rank the suggestions based on their similarity.
- Display the suggestions clearly to the user.

The project also considers the need for supporting different languages. The system uses separate dictionaries for **English, Hindi, and Telugu**, along with Unicode-based word processing.

3. Proposed Solution

The proposed system is a web-based intelligent spell checker. It consists of a frontend and a Java backend.

The user enters a word, sentence, or paragraph into the application. The user can also select the required language. The system then processes the input and checks each word against the selected dictionary.

The general working process is:

User Input → Language Selection → Dictionary Lookup → Detect Incorrect Word → Calculate Levenshtein Distance → Rank Suggestions → Display Correction

If the word is found in the dictionary, it is treated as a correctly spelled word.

If the word is not found, the system calculates the Levenshtein distance between the incorrect word and dictionary words. Words with smaller distances are considered better matches and are displayed as suggestions.

For example:

Input: studnet

Possible suggestion:

student

The system can also process complete sentences and paragraphs instead of checking only a single word.

4. Levenshtein Edit Distance

Levenshtein Edit Distance is a string comparison algorithm used to calculate the minimum number of single-character operations required to transform one string into another.

The three operations are:

1. **Insertion** – adding a character.
2. **Deletion** – removing a character.
3. **Substitution** – replacing one character with another.

For example:

Word 1: cat

Word 2: bat

Only one character needs to be substituted:

$c \rightarrow b$

Therefore, the edit distance is **1**.

Another example is:

Word 1: studnet

Word 2: student

The characters are rearranged through a small number of edits, making student a close dictionary match. A smaller edit distance indicates that two words are more similar. The spell checker uses this property to identify suitable correction suggestions. where

the cost is 0 when the characters are equal and 1 when they are different.

This demonstrates the practical use of Dynamic Programming in the project.

ALGORITHM / PSEUDO CODE

1. Overview of the Algorithm

The proposed Intelligent Spell Checker accepts a word, sentence, or paragraph from the user and identifies incorrectly spelled words. For every word, the system first checks whether it exists in the selected language dictionary. If the word is present, it is considered correctly spelled.

If the word is not found in the dictionary, the system calculates the **Levenshtein Edit Distance** between the incorrect word and the words stored in the dictionary. The Levenshtein algorithm calculates the minimum number of character-level operations required to convert one word into another.

The three operations considered are:

1. **Insertion** – adding a character.
2. **Deletion** – removing a character.
3. **Substitution** – replacing one character with another.

The Levenshtein calculation is implemented using **Dynamic Programming**. The calculated distances are then used to rank possible corrections. The closest words are displayed as suggestions to the user.

The complete algorithm can be represented as:

Input Text



Select Language



Load Appropriate Dictionary



Split Text into Words



Check Each Word in Dictionary



If Correct → Continue



If Incorrect → Calculate Levenshtein Distance



Rank Candidate Words



Generate Suggestions



Display Incorrect Words and Suggestions

2. Step-by-Step Algorithm

Step 1: Start the Application

The application is started by running the Java backend. The system initializes the spell checker and loads the available dictionaries.

The dictionaries contain valid words for different supported languages.

Pseudocode

START

Initialize Spell Checker

Load English Dictionary

Load Hindi Dictionary

Load Telugu Dictionary

Start Server

Wait for User Input

Step 2: Select the Language

The user selects the language in which the text has to be checked.

The project supports:

- English
- Hindi
- Telugu

Based on the selected language, the corresponding dictionary is used.

Pseudocode

INPUT selectedLanguage

IF selectedLanguage = English
 dictionary ← English Dictionary

ELSE IF selectedLanguage = Hindi
 dictionary ← Hindi Dictionary

ELSE IF selectedLanguage = Telugu
 dictionary ← Telugu Dictionary

ELSE
 Display "Unsupported Language"

END IF

Step 3: Accept User Input

The system accepts a word, sentence, or paragraph from the user.

For example:

I am studnet and I like programing.

The input can contain multiple words and punctuation marks.

Pseudocode

INPUT text

IF text is empty

 Display "Please enter some text"

 STOP

END IF

3. Tokenization of the Input

After receiving the input, the system identifies individual words.

For example:

Input:

I am studnet and I like programing.

The system identifies:

I

am

studnet

and

I

like

programing

Punctuation such as ., ,, !, and ? is not treated as part of the word.

Pseudocode

words ← Extract words from text

FOR each word IN words

 Process word

END FOR

The system also supports Unicode characters so that words from languages such as Hindi and Telugu can be processed.

4. Dictionary Lookup

Each extracted word is checked against the selected dictionary.

A **HashSet** can be used for dictionary storage because it provides efficient average-case lookup.

For example:

Dictionary:

student
programming
computer
science

Input:

student

Since student exists in the dictionary, it is considered correct.

Input:

studnet

Since studnet does not exist in the dictionary, it is considered a possible spelling mistake.

Pseudocode

FOR each word IN words

 IF dictionary contains word

 Mark word as CORRECT

 ELSE

 Mark word as INCORRECT

Find suggestions

END IF

END FOR

RESULTS

**SpellWise**
Intelligent Spell Checker

DSA • Dynamic Programming

LEVENSHTEIN EDIT DISTANCE

Write. Check. Correct.

Find spelling mistakes and get intelligent suggestions using Levenshtein Edit Distance.

**Write your text**
Enter a word, sentence or paragraph.

GB English

ANALYSIS

Results

4
ERRORS

iam

a (2), at (2), all (3)

studetn

student (2), studying (3)

Words: 0

 Clear Check Spelling

**How it works**

Each word is compared with dictionary words using Levenshtein Edit Distance. The closest words are ranked and displayed as suggestions.

**Smart Suggestions**

Suggestions with an edit distance of up to 3 are displayed. Click an incorrect word to choose a correction.

**Multiple Languages**

Supports English, Hindi and Telugu dictionaries using Unicode text processing.

Intelligent Spell Checker • Java + HTML + CSS + JavaScript • Levenshtein Edit Distance

Find spelling mistakes and get intelligent suggestions using Levenshtein Edit Distance.

**Write your text**
Enter a word, sentence or paragraph.

GB English

tabel

ANALYSIS

1

ERRORS

Results

tabel

table (2), all (3), are (3)